

NATIONAL UNIVERSITY

551 M.F. JHOCSO ST., SAMPALOC, MANILA

COLLEGE OF ENGINEERING

Name/Section	Lance C. Ortiz / COE 232	Rating
Date performed	August 20, 2026	
Date submitted	August 20, 2026	

SOFTWARE DESIGN EXPERIMENT NO. 5

Campus Hardware Inventory System: Authentication, Inventory Management, and Administrative Controls

I. OBJECTIVES:

By the end of this activity, students should be able to:

1. Demonstrate the integrated authentication, role-based access, and account-security functions of the Campus Hardware Inventory System.
2. Perform hardware inventory, user-profile, and administrative account-management operations using the appropriate user roles.
3. Evaluate the system's data validation, security, inventory-management, and logging functions through practical implementation.

II. COURSE LEARNING OUTCOMES (CLOs)

1. Apply software design principles in developing and operating an integrated desktop information system.
2. Implement and demonstrate authentication, validation, role-based access, inventory management, and administrative functions.
3. Evaluate system behavior using functional operations, account records, inventory records, and system logs.

III. SOFTWARE AND TOOLS REQUIRED

- Visual Studio Code (VS Code)
- SQLite DB
- A terminal or command prompt
- SQLite

IV. PROCEDURE & SOURCE CODE

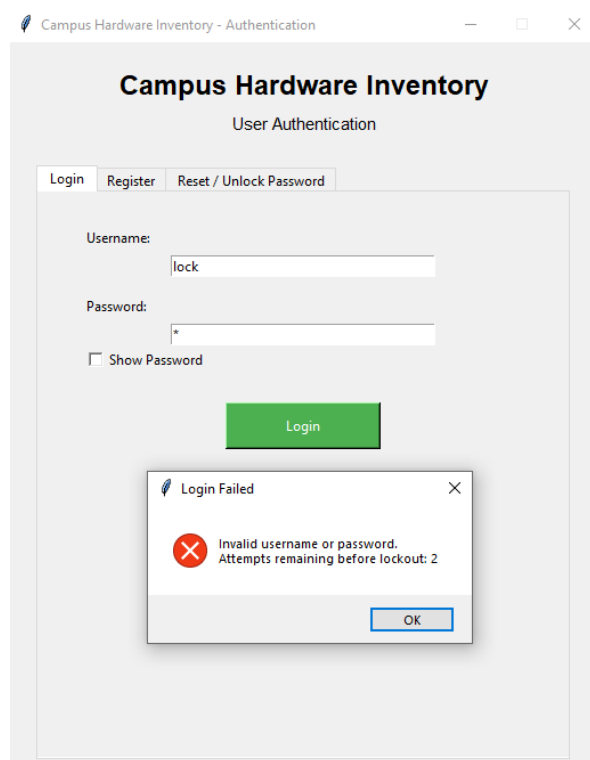
A. Experimental Tasks

Task 1 – User Registration and Authentication

1. Create a user account similar to the previous Lab Experiment 4.
2. Provide the required username, email, password, and user role.
3. Implement email and password validation.
4. Implement password hashing and account security.
5. Log in using the newly created account.

Task 2 – Account Lockout and Password Reset

1. Modify the login system to remove the 30-second countdown.
2. Replace the countdown with an account lockout mechanism.
3. Lock the account after three unsuccessful login attempts.
4. Allow the locked user to proceed to the Reset / Unlock Password function.
5. Submit a password-reset request using the registered email.
6. Log in as an ADMIN.
7. Open Admin Approvals.
8. Approve or reject the password-reset request.



Task 3 – Hardware Inventory Management

1. Log in using an ADMIN account.
2. Open the Hardware Catalog.
3. Add a new hardware component.
4. Provide the required hardware information.
5. Save the hardware record.
6. Implement automatic inventory status updating.
7. Search and filter the hardware inventory.
8. Delete a selected hardware item.
9. Export the inventory records to CSV.

1	=====						
2	CAMPUS HARDWARE INVENTORY MANAGEMENT REPORT						
3	Generated On: 2026-08-20 10:00:06						
4	Total Recorded Items: 10						
5	=====						
6							
7	Item ID	Item Nam	Category	Quantity	Unit Price	Total Price	Status
8	1	Arduino U	Microcont	10	\$50.00	\$500.00	In Stock
9	2	ESP32	Microcont	10	\$20.00	\$200.00	In Stock
10	3	LED (Blue)	LED	50	\$1.00	\$50.00	In Stock
11	4	LED (Gree	LED	50	\$1.00	\$50.00	In Stock
12	5	PIC16F877	PIC	10	\$10.00	\$100.00	In Stock
13	6	PIC92831	PIC	2	\$10.00	\$20.00	Low Stock
14	7	LED (Pink)	LED	50	\$1.00	\$50.00	In Stock
15	8	DC Power	Power Sou	10	\$30.00	\$300.00	In Stock
16	9	9 V Batter	Power Sou	0	\$5.00	\$0.00	Out of Stock
17	10	RX 580	GPU	2	\$400.00	\$800.00	Low Stock
18							
19					GRAND TC	\$2,070.00	
20							

Task 4 – Inventory Record Maintenance

1. Log in using an ADMIN account.
2. Open the Hardware Catalog.
3. Select an existing hardware record.
4. Modify the inventory information as needed.
5. Save the updated inventory record.
6. Review the updated quantity, category, unit price, and inventory status.
7. Search and filter the updated record.

Export the updated inventory records to CSV.

Campus Hardware Inventory - User: Lance (Admin)
Total Asset Value: \$2,070.00

Hardware Catalog
Admin Approvals

Add New Equipment

Name:
Category:
Quantity:
Unit Price (\$):
Save Item

Status Legend

In Stock (> 5)
Low Stock (1-5)
Out of Stock (0)

Filter & Search

Category: All Categories
Search Name:
Clear Filters

ID	Name	Category	Qty	Price	Status
9	9 V Battery	Power Source	0	5.0	Out of Stock
1	Arduino Uno	Microcontroller	10	50.0	In Stock
8	DC Power Supply	Power Source	10	30.0	In Stock
2	ESP32	Microcontroller	10	20.0	In Stock
3	LED (Blue)	LED	50	1.0	In Stock
4	LED (Green)	LED	50	1.0	In Stock
7	LED (Pink)	LED	50	1.0	In Stock
5	PIC16F877A	PIC	10	10.0	In Stock
6	PIC92831	PIC	2	10.0	Low Stock
10	RX 580	GPU	2	400.0	Low Stock

Update / Delete Selected Record

New Quantity:
New Unit Price (\$):
Update Record
Delete Record

Export Inventory to CSV

Logout

1	=====							
2	CAMPUS HARDWARE INVENTORY MANAGEMENT REPORT							
3	Generated On: 2026-08-20 10:02:06							
4	Total Recorded Items: 9							
5	=====							
6								
7	Item ID	Item Nam	Category	Quantity	Unit Price	Total Price	Status	
8	1	Arduino U	Microcont	10	\$50.00	\$500.00	In Stock	
9	2	ESP32	Microcont	10	\$20.00	\$200.00	In Stock	
10	3	LED (Blue)	LED	0	\$1.00	\$0.00	Out of Stock	
11	4	LED (Gree	LED	50	\$1.00	\$50.00	In Stock	
12	5	PIC16F877	PIC	10	\$10.00	\$100.00	In Stock	
13	7	LED (Pink)	LED	4	\$1.00	\$4.00	Low Stock	
14	8	DC Power	Power Sou	10	\$30.00	\$300.00	In Stock	
15	9	9 V Batter	Power Sou	0	\$5.00	\$0.00	Out of Stock	
16	10	RX 580	GPU	2	\$300.00	\$600.00	Low Stock	
17								
18					GRAND TC	\$1,754.00		

Task 5 – User Profile and Security Management

1. Log in using a USER account.
2. Open My Profile & Security.
3. Review the account information and assigned role.
4. Change the account password using the direct password-change function.
5. Log out and log in using the updated password.

[Login](#)
[Register](#)
[Reset / Unlock Password](#)

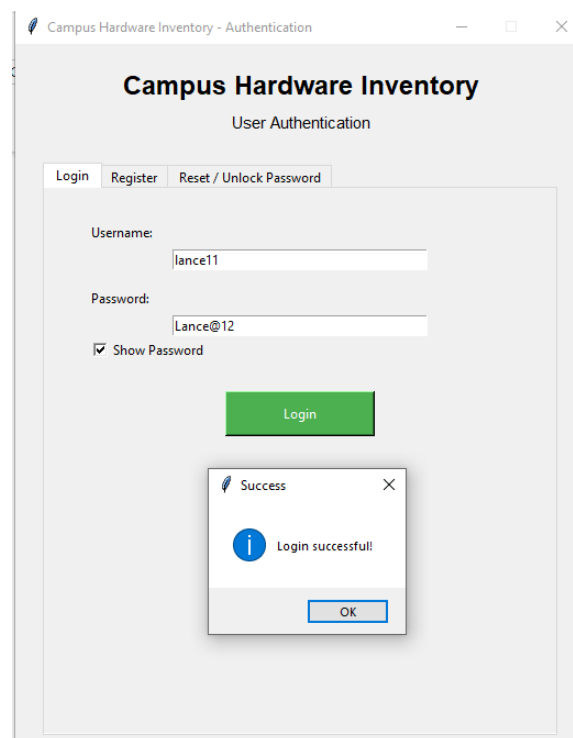
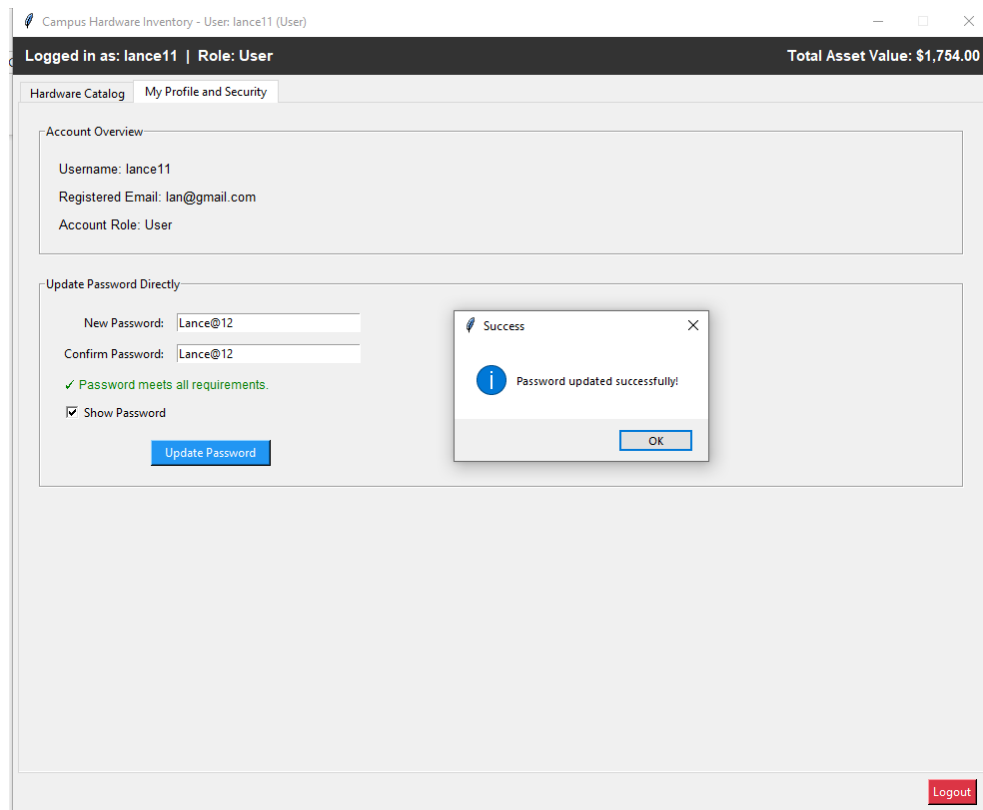
Username:

Password:

☒ Show Password

Success
×

 Login successful!



Task 6 – Administrator User and Account Management

6. Log in using an ADMIN account.
7. Open Admin Approvals.
8. Review pending password-reset requests.
9. Approve or reject selected password-reset requests.
10. Review the updated request status.
11. Verify that an approved reset request unlocks the corresponding account.

Campus Hardware Inventory - Authentication

Campus Hardware Inventory

User Authentication

Login

Register

Reset / Unlock Password

Reset / Unlock your account by submitting a new password. An administrator must approve this request before you can log in again.

Account Username:

lock

Registered Email:

lock@gmail.com

Desired New Password:

Lock@123

✓ Password meets all requirements.

Confirm New Password:

Lock@123

☒ Show Password

Submit Reset Request

Request Submitted

Your reset request has been submitted. An administrator must approve it before you can log in again.

OK

Hardware Catalog

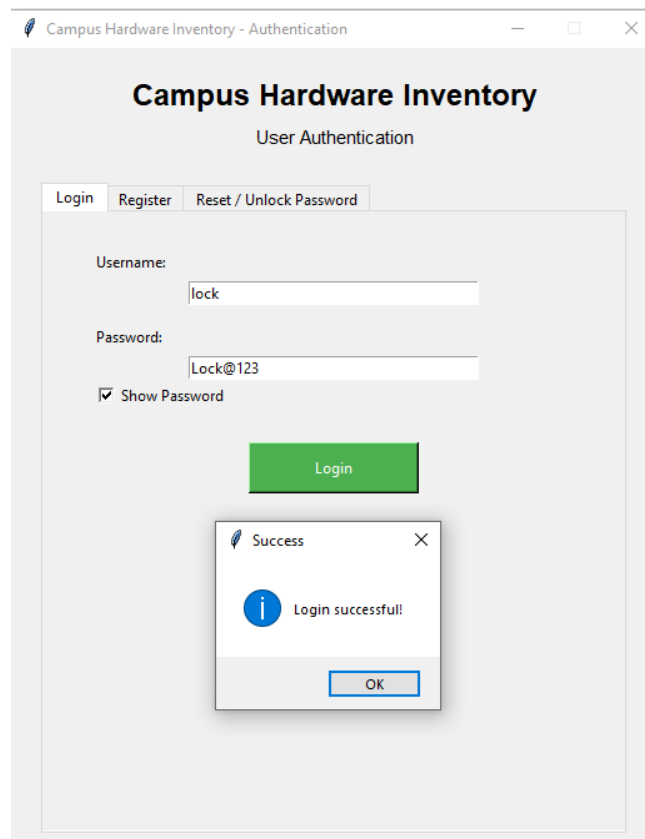
Admin Approvals

Pending Password Reset / Unlock Requests

Request ID	Username	Email	Timestamp
2	lock	lock@gmail.com	2026-08-20 10:06:10

Approve Reset

Reject Reset



Task 7 – System History, Logging, and Records

1. Review the available hardware inventory records.
2. Review the password-reset request records as an ADMIN.
3. Review the account and inventory operations performed during the activity.
4. Open app_logging/app.log.
5. Review the recorded registration, login, logout, password-reset, inventory, and administrative activities.

Campus Hardware Inventory - User: lock (User)

Logged in as: lock | Role: User

Total Asset Value: \$1,754.00

Hardware CatalogMy Profile and Security

Filter & Search

Category: All Categories

Search Name:

Clear Filters

ID	Name	Category	Qty	Price	Status
9	9 V Battery	Power Source	0	5.0	Out of Stock
1	Arduino Uno	Microcontroller	10	50.0	In Stock
8	DC Power Supply	Power Source	10	30.0	In Stock
2	ESP32	Microcontroller	10	20.0	In Stock
3	LED (Blue)	LED	0	1.0	Out of Stock
4	LED (Green)	LED	50	1.0	In Stock
7	LED (Pink)	LED	4	1.0	Low Stock
5	PIC16F877A	PIC	10	10.0	In Stock
10	RX 580	GPU	2	300.0	Low Stock

Export Inventory to CSV

Logout

Campus Hardware Inventory - User: Lance (Admin)

Logged in as: Lance | Role: Admin

Total Asset Value: \$1,754.00

Hardware CatalogAdmin Approvals

Pending Password Reset / Unlock Requests

Request ID	Username	Email	Timestamp
------------	----------	-------	-----------

Approve ResetReject Reset

Logout

```
File Edit Format View Help
2026-08-20 09:02:33 - [INFO] - CampusHardwareLogger - User 'lance11' updated their password directly.
2026-08-20 09:02:39 - [INFO] - CampusHardwareLogger - CSV Export generated: inventory_report.csv. Total Value: $2,070.00
2026-08-20 09:02:50 - [INFO] - CampusHardwareLogger - User 'lance11' logged out.
2026-08-20 09:03:21 - [INFO] - CampusHardwareLogger - Account created: 'lock' with role 'User'.
2026-08-20 09:03:28 - [INFO] - CampusHardwareLogger - User 'lock' logged in successfully as 'User'.
2026-08-20 09:04:01 - [INFO] - CampusHardwareLogger - User 'lock' logged out.
2026-08-20 09:04:13 - [WARNING] - CampusHardwareLogger - Failed login attempt for 'lock'. Attempts: 1
2026-08-20 09:04:18 - [WARNING] - CampusHardwareLogger - Failed login attempt for unknown user 'lance15'
2026-08-20 09:04:24 - [WARNING] - CampusHardwareLogger - Failed login attempt for 'lock'. Attempts: 2
2026-08-20 09:04:27 - [WARNING] - CampusHardwareLogger - Account 'lock' locked after 3 failed attempts.
2026-08-20 09:04:34 - [WARNING] - CampusHardwareLogger - Login blocked for locked account 'lock'.
2026-08-20 09:05:06 - [WARNING] - CampusHardwareLogger - Reset request failed: email mismatch for 'lock'.
2026-08-20 09:05:22 - [INFO] - CampusHardwareLogger - Password reset/unlock request submitted for 'lock'.
2026-08-20 09:05:30 - [WARNING] - CampusHardwareLogger - Login blocked for locked account 'lock'.
2026-08-20 09:05:39 - [WARNING] - CampusHardwareLogger - Failed login attempt for unknown user 'lance15'
2026-08-20 09:05:46 - [WARNING] - CampusHardwareLogger - Failed login attempt for unknown user 'lance15'
2026-08-20 09:05:49 - [INFO] - CampusHardwareLogger - User 'lance' logged in successfully as 'Admin'.
2026-08-20 09:05:55 - [INFO] - CampusHardwareLogger - Reset request #1 for 'lock' approved. Account unlocked.
2026-08-20 09:06:02 - [INFO] - CampusHardwareLogger - User 'lance' logged out.
2026-08-20 09:06:12 - [INFO] - CampusHardwareLogger - User 'lock' logged in successfully as 'User'.
2026-08-20 09:57:27 - [INFO] - CampusHardwareLogger - Database initialized successfully.
2026-08-20 09:57:58 - [WARNING] - CampusHardwareLogger - Failed login attempt for 'lock'. Attempts: 1
2026-08-20 09:59:02 - [INFO] - CampusHardwareLogger - User 'lance' logged in successfully as 'Admin'.
2026-08-20 10:00:06 - [INFO] - CampusHardwareLogger - CSV Export generated: inventory_report.csv. Total Value: $2,070.00
2026-08-20 10:01:37 - [INFO] - CampusHardwareLogger - Hardware ID 3 updated. New Qty: 0, New Price: 1.0
2026-08-20 10:01:43 - [INFO] - CampusHardwareLogger - Hardware ID 7 updated. New Qty: 4, New Price: 1.0
2026-08-20 10:01:51 - [INFO] - CampusHardwareLogger - Hardware ID 10 updated. New Qty: 2, New Price: 300.0
2026-08-20 10:02:01 - [INFO] - CampusHardwareLogger - Hardware ID 6 ('PIC92831') deleted.
2026-08-20 10:02:06 - [INFO] - CampusHardwareLogger - CSV Export generated: inventory_report.csv. Total Value: $1,754.00
2026-08-20 10:02:48 - [INFO] - CampusHardwareLogger - User 'lance' logged out.
2026-08-20 10:03:04 - [INFO] - CampusHardwareLogger - User 'lance11' logged in successfully as 'User'.
2026-08-20 10:04:00 - [INFO] - CampusHardwareLogger - User 'lance11' updated their password directly.
2026-08-20 10:04:34 - [INFO] - CampusHardwareLogger - User 'lance11' logged out.
2026-08-20 10:04:48 - [INFO] - CampusHardwareLogger - User 'lance11' logged in successfully as 'User'.
2026-08-20 10:05:36 - [INFO] - CampusHardwareLogger - User 'lance11' logged out.
2026-08-20 10:06:10 - [INFO] - CampusHardwareLogger - Password reset/unlock request submitted for 'lock'.
2026-08-20 10:07:03 - [INFO] - CampusHardwareLogger - User 'lance' logged in successfully as 'Admin'.
2026-08-20 10:07:30 - [INFO] - CampusHardwareLogger - Reset request #2 for 'lock' approved. Account unlocked.
2026-08-20 10:07:35 - [INFO] - CampusHardwareLogger - User 'lance' logged out.
2026-08-20 10:07:43 - [INFO] - CampusHardwareLogger - User 'lock' logged in successfully as 'User'.
2026-08-20 10:08:52 - [INFO] - CampusHardwareLogger - User 'lock' logged out.
2026-08-20 10:09:00 - [INFO] - CampusHardwareLogger - User 'lance' logged in successfully as 'Admin'.
```

V. GUIDE QUESTIONS

1. How does role-based access control prevent unauthorized users from performing administrative operations?

Each account gets tagged as either User or Admin when it's created, and. A User's interface simply doesn't include the Add/Update/Delete buttons or the Admin tab, they're not there to click. So there's no "getting around" it, because the option was never given in the first place.

2. How does the three-attempt account-lockout mechanism improve system security?

It stops someone from sitting there guessing passwords. Three wrong tries and the account locks completely, even the correct password won't work until an admin actually approves a reset.

3. Why is password hashing important when storing user credentials?

Because we never want real passwords sitting in a database. We hash them with bcrypt before saving, so what's stored can't be turned back into the actual password.

4. How does automatic inventory-status calculation improve inventory management?

Instead of someone manually deciding "this is low stock," the status is calculated straight from quantity every time. Also better efficiency and automation to the overall system.

5. How does input validation improve the reliability and security of the system?

It filters bad data before it ever reaches the database valid emails, strong passwords, no negative numbers. So no incorrect data would be stored.

6. Why are system logs and administrative records important in a hardware inventory system?

Without logs, you're just guessing what happened. The log file timestamps logins, failed attempts, lockouts, and inventory changes, so you can actually trace things back. The reset-request table works the same way a record of who got unlocked.

VI. CONCLUSION

Write a short discussion explaining how the Campus Hardware Inventory System integrates authentication, account security, hardware inventory management, user profile functions, administrative controls, and system logging into one desktop application. Include the results of the laboratory activity and identify any problems encountered during implementation.

Building this system showed how much authentication, account security, inventory management, user profiles, admin controls, and logging all depend on each other. Registering both roles, triggering a lockout, submitting a reset, and approving it as admin all went as expected, with the User and Admin interfaces staying properly separated. However, the hardest part of the whole project was adding the Admin role, since so much ended up tied to it namely the login flow, what buttons even showed up, the approval process, all of it had to be rethought once there were two different kinds of users instead of one. Overall, it was a good reminder that one new feature like a role can touch almost every part of an app, not just the part it was meant for.

```
import sqlite3

import os

import csv

import logging

import re

import time

from datetime import datetime

import tkinter as tk

from tkinter import ttk, messagebox

from pydantic import BaseModel, Field, field_validator, ValidationError

import bcrypt
```

```

# =====

# 1. CENTRAL LOGGING SETUP

# =====

def setup_logger():

    log_dir = "app_logging"

    if not os.path.exists(log_dir):

        os.makedirs(log_dir)

    logging.basicConfig(

        filename=os.path.join(log_dir, "app.log"),

        level=logging.INFO,

        format="%(asctime)s - [%(levelname)s] - %(name)s - %(message)s",

        datefmt="%Y-%m-%d %H:%M:%S"

    )

    return logging.getLogger("CampusHardwareLogger")

logger = setup_logger()

# =====

# 2. DATABASE INITIALIZATION

# =====

DB_NAME = "hardware_inventory.db"

ROLE_USER = "User"

```

```
ROLE_ADMIN = "Admin"
```

```
VALID_ROLES = (ROLE_USER, ROLE_ADMIN)
```

```
def init_db(db_name=DB_NAME):
```

```
    try:
```

```
        conn = sqlite3.connect(db_name)
```

```
        cursor = conn.cursor()
```

```
        # -----
```

```
        # Users Table
```

```
        # -----
```

```
        cursor.execute("""
```

```
            CREATE TABLE IF NOT EXISTS users (
```

```
                id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
                username TEXT UNIQUE NOT NULL,
```

```
                email TEXT,
```

```
                password_hash TEXT NOT NULL,
```

```
                failed_attempts INTEGER DEFAULT 0,
```

```
                locked_until REAL DEFAULT 0
```

```
            )
```

```
        """)
```

```
        # -----
```

```
        # Database Migration
```

```
        # -----
```

```
        # This allows older databases to work even if
```

```
        # they were created using the original schema.
```

```
        cursor.execute("PRAGMA table_info(users)")
```

```
existing_columns = [column[1] for column in cursor.fetchall()]
```

```
if "email" not in existing_columns:
```

```
    cursor.execute("ALTER TABLE users ADD COLUMN email TEXT")
```

```
    logger.info("Database migration: Added email column.")
```

```
if "failed_attempts" not in existing_columns:
```

```
    cursor.execute(
```

```
        "ALTER TABLE users ADD COLUMN failed_attempts INTEGER DEFAULT 0"
```

```
    )
```

```
    logger.info("Database migration: Added failed_attempts column.")
```

```
if "locked_until" not in existing_columns:
```

```
    cursor.execute(
```

```
        "ALTER TABLE users ADD COLUMN locked_until REAL DEFAULT 0"
```

```
    )
```

```
    logger.info("Database migration: Added locked_until column.")
```

```
if "role" not in existing_columns:
```

```
    cursor.execute(
```

```
        "ALTER TABLE users ADD COLUMN role TEXT DEFAULT 'User'"
```

```
    )
```

```
    logger.info("Database migration: Added role column.")
```

```
if "locked" not in existing_columns:
```

```
    cursor.execute(
```

```
        "ALTER TABLE users ADD COLUMN locked INTEGER DEFAULT 0"
```

```
    )
```

```
    logger.info("Database migration: Added locked column.")
```

```
# -----
```

Hardware Inventory Table

cursor.execute("""

```
CREATE TABLE IF NOT EXISTS hardware (  
    item_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    item_name TEXT UNIQUE NOT NULL,  
    category TEXT NOT NULL,  
    quantity INTEGER NOT NULL,  
    unit_price REAL NOT NULL,  
    status TEXT NOT NULL  
)
```

""")

Password Reset / Unlock Requests Table

cursor.execute("""

```
CREATE TABLE IF NOT EXISTS reset_requests (  
    request_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    username TEXT NOT NULL,  
    email TEXT NOT NULL,  
    new_password_hash TEXT NOT NULL,  
    timestamp TEXT NOT NULL,  
    status TEXT NOT NULL DEFAULT 'pending'  
)
```

""")

conn.commit()

conn.close()

```
logger.info("Database initialized successfully.")
```

```
except sqlite3.Error as e:
```

```
    logger.error(f"Database setup error: {e}")
```

```
# =====
```

```
# 3. PYDANTIC SCHEMAS & VALIDATION
```

```
# =====
```

```
def validate_password_rules(v):
```

```
    """Shared password rule validator used by every schema that collects  
    a new password (registration and password reset/update)."""
```

```
    if len(v) < 8:
```

```
        raise ValueError(  
            "Password must contain at least 8 characters."  
        )
```

```
    if not re.search(r"[A-Z]", v):
```

```
        raise ValueError(  
            "Password must contain at least one uppercase letter."  
        )
```

```
    if not re.search(r"[0-9]", v):
```

```
        raise ValueError(  
            "Password must contain at least one number."  
        )
```

```
    if not re.search(r"[@#$%^&*]", v):
```

```
raise ValueError(
    "Password must contain at least one special character (@#$$%^&*)."
)
```

```
return v
```

```
class UserRegisterSchema(BaseModel):
```

```
    username: str = Field(..., min_length=3, max_length=20)
```

```
    email: str
```

```
    password: str
```

```
    role: str
```

```
# -----
```

```
# Username Validation
```

```
# -----
```

```
@field_validator("username")
```

```
@classmethod
```

```
def username_alphanumeric(cls, v):
```

```
    if not re.match(r"^[a-zA-Z0-9_]+$", v):
```

```
        raise ValueError(
```

```
            "Username must contain only letters, numbers, and underscores."
```

```
        )
```

```
    return v
```

```
# -----
```

```
# Email Validation
```

```
# -----
```

```

@field_validator("email")
@classmethod
def validate_email(cls, v):
    v = v.strip().lower()

    if not re.match(
        r"^[a-zA-Z0-9._%+-]+@(gmail\.com|yahoo\.com)$",
        v
    ):
        raise ValueError(
            "Email must be a valid @gmail.com or @yahoo.com address."
        )

    return v

```

```

# -----
# Password Validation
# -----

```

```

@field_validator("password")
@classmethod
def validate_password(cls, v):
    return validate_password_rules(v)

```

```

# -----
# Role Validation
# -----

```

```

@field_validator("role")
@classmethod
def validate_role(cls, v):

```

```
if v not in VALID_ROLES:

    raise ValueError(

        f"Role must be one of: {' '.join(VALID_ROLES)}."

    )
```

```
return v
```

```
class PasswordResetSchema(BaseModel):
```

```
    username: str = Field(..., min_length=1)
```

```
    email: str
```

```
    password: str
```

```
    @field_validator("email")
```

```
    @classmethod
```

```
    def validate_email(cls, v):
```

```
        v = v.strip().lower()
```

```
        if not re.match(
```

```
            r"^[a-zA-Z0-9._%+-]+@(gmail\.com|yahoo\.com)$",
```

```
            v
```

```
        ):
```

```
            raise ValueError(
```

```
                "Email must be a valid @gmail.com or @yahoo.com address."
```

```
            )
```

```
        return v
```

```
    @field_validator("password")
```

```
    @classmethod
```

```
    def validate_password(cls, v):
```

```
    return validate_password_rules(v)
```

```
class HardwareSchema(BaseModel):
```

```
    item_name: str = Field(..., min_length=2, max_length=100)
```

```
    category: str = Field(..., min_length=2, max_length=50)
```

```
    quantity: int = Field(..., ge=0)
```

```
    unit_price: float = Field(..., ge=0.0)
```

```
def calculate_status(quantity: int) -> str:
```

```
    if quantity > 5:
```

```
        return "In Stock"
```

```
    elif 1 <= quantity <= 5:
```

```
        return "Low Stock"
```

```
    else:
```

```
        return "Out of Stock"
```

```
# =====
```

```
# 4. PASSWORD REQUIREMENT CHECKER
```

```
# =====
```

```
def get_password_requirements(password):
```

```
    """
```

```
    Returns a list containing the password requirements
```

```
    that have not yet been satisfied.
```

```
    """
```

```

missing = []

if len(password) < 8:
    missing.append("at least 8 characters")

if not re.search(r"[A-Z]", password):
    missing.append("one uppercase letter")

if not re.search(r"[0-9]", password):
    missing.append("one number")

if not re.search(r"[@#$%^&*]", password):
    missing.append("one special character (@#$%^&*)")

return missing

```

```

PASSWORD_REQUIREMENTS_TEXT = (
    "Password requirements:\n"
    "• At least 8 characters\n"
    "• One uppercase letter\n"
    "• One number\n"
    "• One special character (@#$%^&*)"
)

```

```

# =====
# 5. CONTROLLER / BACKEND LOGIC
# =====

```

```
class InventoryController:
```

```
    MAX_FAILED_ATTEMPTS = 3
```

```
    def __init__(self, db_name=DB_NAME):
```

```
        self.db_name = db_name
```

```
    # =====
```

```
    # USER REGISTRATION
```

```
    # =====
```

```
    def register_user(self, username, email, password, role):
```

```
        try:
```

```
            validated = UserRegisterSchema(
```

```
                username=username,
```

```
                email=email,
```

```
                password=password,
```

```
                role=role
```

```
            )
```

```
        except ValidationError as e:
```

```
            err_msg = e.errors()[0]["msg"]
```

```
            logger.warning(
```

```
                f"Registration validation failure: {err_msg}"
```

```
            )
```

```
        return False, f"Validation Error: {err_msg}"
```

```
conn = None
```

```
try:
```

```
    conn = sqlite3.connect(self.db_name)
```

```
    cursor = conn.cursor()
```

```
    cursor.execute(
```

```
        "SELECT username, email FROM users WHERE LOWER(username) = LOWER(?) OR  
        LOWER(email) = LOWER(?)",
```

```
        (validated.username, validated.email)
```

```
    )
```

```
    existing_user = cursor.fetchone()
```

```
    if existing_user:
```

```
        logger.warning(
```

```
            f"Registration attempt failed for '{validated.username}' / '{validated.email}' because the  
            account already exists."
```

```
        )
```

```
        return False, "Username or email already registered. Please use different details or log in."
```

```
    hashed_pw = bcrypt.hashpw(
```

```
        validated.password.encode("utf-8"),
```

```
        bcrypt.gensalt()
```

```
    )
```

```
    cursor.execute("""
```

```
        INSERT INTO users
```

```
        (
```

```
            username,
```

```
            email,
```

```
            password_hash,
```

```

        failed_attempts,
        locked_until,
        role,
        locked
    )
VALUES (?, ?, ?, 0, 0, ?, 0)
"""', (
    validated.username,
    validated.email,
    hashed_pw.decode("utf-8"),
    validated.role
))

conn.commit()

logger.info(
    f"Account created: '{validated.username}' with role '{validated.role}'"
)

return True, "Registration successful! You may now log in."

except sqlite3.IntegrityError:

    return False, "Username or email already registered. Please use different details."

except sqlite3.Error as e:

    logger.error(
        f"Registration database error: {e}"
    )

```

```
    return False, "Database error occurred during registration. Please try again."
```

```
except Exception as e:
```

```
    logger.error(
        f"Unexpected registration error: {e}"
    )
```

```
    return False, "An unexpected error occurred during registration. Please try again."
```

```
finally:
```

```
    if conn is not None:
```

```
        conn.close()
```

```
# =====
```

```
# LOGIN
```

```
# =====
```

```
def login_user(self, username, password):
```

```
    """
```

```
    Returns a tuple of (status, message, role).
```

```
    status is one of:
```

```
        "success" - credentials are correct
```

```
        "locked" - account is locked and needs a password reset
```

```
        "fail" - invalid credentials / unknown user
```

```
    """
```

```
    if not username or not password:
```

```
return "fail", "Please enter both username and password.", None
```

```
conn = sqlite3.connect(self.db_name)
```

```
cursor = conn.cursor()
```

```
cursor.execute("""
```

```
    SELECT
```

```
        password_hash,
```

```
        failed_attempts,
```

```
        role,
```

```
        locked
```

```
    FROM users
```

```
    WHERE username = ?
```

```
""", (username,))
```

```
row = cursor.fetchone()
```

```
# -----
```

```
# User Does Not Exist
```

```
# -----
```

```
if not row:
```

```
    conn.close()
```

```
    logger.warning(
```

```
        f"Failed login attempt for unknown user '{username}'"
```

```
    )
```

```
return "fail", "Invalid username or password.", None
```

```
password_hash = row[0]
failed_attempts = row[1] or 0
role = row[2] or ROLE_USER
locked = row[3] or 0
```

```
# =====
# ACCOUNT LOCKED - NEEDS RESET / UNLOCK
# =====
```

```
if locked:
```

```
    conn.close()
```

```
    logger.warning(
        f"Login blocked for locked account '{username}'."
    )
```

```
    return (
        "locked",
        "Your account is locked due to too many failed login "
        "attempts.\nPlease reset your password to unlock it.",
        None
    )
```

```
# =====
# CHECK PASSWORD
# =====
```

```
password_correct = bcrypt.checkpw(
    password.encode("utf-8"),
    password_hash.encode("utf-8")
```

)

=====

SUCCESSFUL LOGIN

=====

if password_correct:

```
cursor.execute("""
    UPDATE users
    SET failed_attempts = 0
    WHERE username = ?
""", (username,))
```

conn.commit()

conn.close()

```
logger.info(
    f"User '{username}' logged in successfully as '{role}'."
)
```

return "success", "Login successful!", role

=====

FAILED LOGIN

=====

failed_attempts += 1

if failed_attempts >= self.MAX_FAILED_ATTEMPTS:

```
cursor.execute("""
    UPDATE users
    SET failed_attempts = ?,
        locked = 1
    WHERE username = ?
""", (
    failed_attempts,
    username
))
```

```
conn.commit()
conn.close()
```

```
logger.warning(
    f"Account '{username}' locked after "
    f"{failed_attempts} failed attempts."
)
```

```
return (
    "locked",
    "Too many failed login attempts.\n"
    "Your account has been locked. Please reset your "
    "password to unlock it.",
    None
)
```

else:

```
cursor.execute("""
    UPDATE users
    SET failed_attempts = ?
```

```

        WHERE username = ?
        """ , (
            failed_attempts,
            username
        ))

    conn.commit()
    conn.close()

    remaining_attempts = (
        self.MAX_FAILED_ATTEMPTS - failed_attempts
    )

```

```

    logger.warning(
        f"Failed login attempt for '{username}'. "
        f"Attempts: {failed_attempts}"
    )

```

```

    return (
        "fail",
        "Invalid username or password.\n"
        f"Attempts remaining before lockout: "
        f"{remaining_attempts}",
        None
    )

```

```

# =====
# PASSWORD RESET / UNLOCK REQUEST
# =====

```

```

def submit_reset_request(self, username, email, new_password, confirm_password):

```

```
username = (username or "").strip()
email_input = (email or "").strip()

if not username:
    return False, "Please enter the account username."

conn = sqlite3.connect(self.db_name)
cursor = conn.cursor()

cursor.execute(
    "SELECT email FROM users WHERE username = ?",
    (username,)
)

row = cursor.fetchone()

# Username must exist first - highest priority error.

if not row:
    conn.close()
    logger.warning(
        f"Reset request failed: unknown username '{username}'"
    )
    return False, "No account found with that username."

registered_email = (row[0] or "").strip().lower()

# Email must match the registered email - second priority error.
# Basic email format check before comparing to the record.
```

```
if not re.match(
    r"^[a-zA-Z0-9._%+-]+@(gmail\.com|yahoo\.com)$",
    email_input.strip().lower()
):
    conn.close()
    return False, "Email must be a valid @gmail.com or @yahoo.com address."
```

```
if email_input.strip().lower() != registered_email:
    conn.close()
    logger.warning(
        f"Reset request failed: email mismatch for '{username}'"
    )
    return False, "The email does not match our records for this username."
```

Password requirements - third priority.

```
try:
    validated = PasswordResetSchema(
        username=username,
        email=email_input,
        password=new_password
    )
except ValidationError as e:
    conn.close()
    err_msg = e.errors()[0]["msg"]
    return False, f"Validation Error: {err_msg}"
```

Confirm password match - final check.

```
if new_password != confirm_password:
    conn.close()
```

```
return False, "New password and confirmation do not match."
```

```
hashed_pw = bcrypt.hashpw(  
    validated.password.encode("utf-8"),  
    bcrypt.gensalt()  
)
```

```
timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```
cursor.execute("""  
    INSERT INTO reset_requests  
    (  
        username,  
        email,  
        new_password_hash,  
        timestamp,  
        status  
    )  
    VALUES (?, ?, ?, ?, 'pending')
```

```
""", (  
    username,  
    registered_email,  
    hashed_pw.decode("utf-8"),  
    timestamp  
)  
)
```

```
conn.commit()
```

```
conn.close()
```

```
logger.info(  
    f"Password reset/unlock request submitted for '{username}'."
```

)

return True, (

"Your reset request has been submitted.\n"

"An administrator must approve it before you can log in again."

)

def get_pending_reset_requests(self):

conn = sqlite3.connect(self.db_name)

cursor = conn.cursor()

cursor.execute("""

SELECT request_id, username, email, timestamp

FROM reset_requests

WHERE status = 'pending'

ORDER BY request_id ASC

""")

rows = cursor.fetchall()

conn.close()

return rows

def approve_reset_request(self, request_id):

conn = sqlite3.connect(self.db_name)

cursor = conn.cursor()

cursor.execute("""

SELECT username, new_password_hash, status

```

        FROM reset_requests

        WHERE request_id = ?

        """ , (request_id,))

row = cursor.fetchone()

if not row:

    conn.close()

    return False, "Request not found."

username, new_password_hash, status = row

if status != "pending":

    conn.close()

    return False, "This request has already been processed."

cursor.execute("""

    UPDATE users

    SET password_hash = ?,

        failed_attempts = 0,

        locked = 0

    WHERE username = ?

    """ , (new_password_hash, username))

cursor.execute("""

    UPDATE reset_requests

    SET status = 'approved'

    WHERE request_id = ?

    """ , (request_id,))

conn.commit()

```

```
conn.close()
```

```
logger.info(
```

```
    f"Reset request #{request_id} for '{username}' approved. Account unlocked."
```

```
)
```

```
return True, f"Reset request for '{username}' approved. Account unlocked."
```

```
def reject_reset_request(self, request_id):
```

```
    conn = sqlite3.connect(self.db_name)
```

```
    cursor = conn.cursor()
```

```
    cursor.execute("""
```

```
        SELECT username, status
```

```
        FROM reset_requests
```

```
        WHERE request_id = ?
```

```
    """, (request_id,))
```

```
    row = cursor.fetchone()
```

```
    if not row:
```

```
        conn.close()
```

```
        return False, "Request not found."
```

```
    username, status = row
```

```
    if status != "pending":
```

```
        conn.close()
```

```
        return False, "This request has already been processed."
```

```

cursor.execute("""
    UPDATE reset_requests
    SET status = 'rejected'
    WHERE request_id = ?
""", (request_id,))

conn.commit()

conn.close()

logger.info(
    f"Reset request #{request_id} for '{username}' rejected."
)

return True, f"Reset request for '{username}' rejected. Account remains locked."

# =====
# DIRECT PASSWORD UPDATE (LOGGED-IN USER)
# =====

def update_password_direct(self, username, new_password, confirm_password):

    if new_password != confirm_password:
        return False, "New password and confirmation do not match."

    missing = get_password_requirements(new_password)

    if missing:
        return False, "Password is missing: " + ", ".join(missing) + "."

    hashed_pw = bcrypt.hashpw(
        new_password.encode("utf-8"),

```

```
        bcrypt.gensalt()
    )
```

```
conn = sqlite3.connect(self.db_name)
cursor = conn.cursor()
```

```
cursor.execute("""
    UPDATE users
    SET password_hash = ?
    WHERE username = ?
""", (hashed_pw.decode("utf-8"), username))
```

```
conn.commit()
conn.close()
```

```
logger.info(f"User '{username}' updated their password directly.")
```

```
return True, "Password updated successfully!"
```

```
# =====
# ACCOUNT / PROFILE INFO
# =====
```

```
def get_user_profile(self, username):
```

```
    conn = sqlite3.connect(self.db_name)
    cursor = conn.cursor()
```

```
    cursor.execute("""
        SELECT username, email, role
        FROM users
```

```

        WHERE username = ?

        """ , (username,))

    row = cursor.fetchone()

    conn.close()

    return row

# =====
# FETCH HARDWARE
# =====

def fetch_all_hardware(self):

    conn = sqlite3.connect(self.db_name)

    cursor = conn.cursor()

    cursor.execute("""
        SELECT
            item_id,
            item_name,
            category,
            quantity,
            unit_price,
            status
        FROM hardware
    """)

    rows = cursor.fetchall()

    conn.close()

```

```
return rows
```

```
# =====
```

```
# FETCH / FILTER HARDWARE (CATEGORY + SEARCH)
```

```
# =====
```

```
def fetch_filtered_hardware(self, category="All Categories", search_text=""):
```

```
    conn = sqlite3.connect(self.db_name)
```

```
    cursor = conn.cursor()
```

```
    query = """
```

```
        SELECT
```

```
            item_id,
```

```
            item_name,
```

```
            category,
```

```
            quantity,
```

```
            unit_price,
```

```
            status
```

```
        FROM hardware
```

```
        WHERE 1 = 1
```

```
    """
```

```
    params = []
```

```
    if category and category != "All Categories":
```

```
        query += " AND category = ?"
```

```
        params.append(category)
```

```
    if search_text:
```

```
query += " AND LOWER(item_name) LIKE ?"
```

```
params.append(f"%{search_text.strip().lower()}%")
```

```
query += " ORDER BY item_name ASC"
```

```
cursor.execute(query, params)
```

```
rows = cursor.fetchall()
```

```
conn.close()
```

```
return rows
```

```
def get_distinct_categories(self):
```

```
    conn = sqlite3.connect(self.db_name)
```

```
    cursor = conn.cursor()
```

```
    cursor.execute("""
```

```
        SELECT DISTINCT category
```

```
        FROM hardware
```

```
        ORDER BY category ASC
```

```
    """)
```

```
    rows = [r[0] for r in cursor.fetchall()]
```

```
    conn.close()
```

```
    return rows
```

```
# =====
```

```
# TOTAL ASSET VALUE
```

```
# =====
```

```
def get_total_asset_value(self):
```

```
    conn = sqlite3.connect(self.db_name)
```

```
    cursor = conn.cursor()
```

```
    cursor.execute("""
```

```
        SELECT SUM(quantity * unit_price)
```

```
        FROM hardware
```

```
    """)
```

```
    total = cursor.fetchone()[0]
```

```
    conn.close()
```

```
    return total if total is not None else 0.0
```

```
# =====
```

```
# ADD HARDWARE
```

```
# =====
```

```
def addHardware(
```

```
    self,
```

```
    item_name,
```

```
    category,
```

```
    quantity_str,
```

```
    unit_price_str
```

```
):
```

try:

qty = int(quantity_str)

price = float(unit_price_str)

validated = HardwareSchema(

item_name=item_name,

category=category,

quantity=qty,

unit_price=price

)

except (ValueError, ValidationError) as e:

if isinstance(e, ValidationError):

err_msg = e.errors()[0]["msg"]

else:

err_msg = "Invalid numbers."

logger.warning(

f"Hardware validation failed: {err_msg}"

)

return False, f"Validation Error: {err_msg}"

status = calculate_status(validated.quantity)

conn = None

try:

```

conn = sqlite3.connect(self.db_name)

cursor = conn.cursor()

# Case-insensitive duplicate check

cursor.execute("""
    SELECT item_id
    FROM hardware
    WHERE LOWER(item_name) = LOWER(?)
""", (
    validated.item_name.strip(),
))

if cursor.fetchone():

    logger.warning(
        f"Duplicate entry attempt: "
        f"{validated.item_name}"
    )

    return False, (
        f"An item named '{validated.item_name}' "
        f"already exists in inventory."
    )

cursor.execute("""
    INSERT INTO hardware
    (
        item_name,
        category,
        quantity,

```

```

        unit_price,

        status
    )
VALUES (?, ?, ?, ?, ?)
"""', (
    validated.item_name.strip(),
    validated.category.strip(),
    validated.quantity,
    validated.unit_price,
    status
))

conn.commit()

logger.info(
    f"New equipment added: '{validated.item_name}'"
)

return True, "Equipment added successfully!"

except sqlite3.IntegrityError:

    logger.warning(
        f"Integrity violation adding hardware: "
        f"'{validated.item_name}'"
    )

    return False, (
        f"An item named '{validated.item_name}' "
        f"already exists!"
    )

```

```
except sqlite3.Error as e:
```

```
    logger.error(  
        f"Database error on adding hardware: {e}"  
    )
```

```
    return False, f"Database Error: {e}"
```

```
finally:
```

```
    if conn:  
        conn.close()
```

```
# =====
```

```
# UPDATE HARDWARE
```

```
# =====
```

```
def update_hardware(  
    self,  
    item_id,  
    quantity_str,  
    unit_price_str  
):
```

```
    conn = None
```

```
    try:
```

```
        conn = sqlite3.connect(self.db_name)
```

```
        cursor = conn.cursor()
```

```

cursor.execute("""
    SELECT quantity, unit_price
    FROM hardware
    WHERE item_id = ?
""", (item_id,))

current_record = cursor.fetchone()

if not current_record:

    return False, (
        "Selected record could not be found."
    )

current_qty, current_price = current_record

new_qty = (
    int(quantity_str)
    if quantity_str.strip()
    else current_qty
)

new_price = (
    float(unit_price_str)
    if unit_price_str.strip()
    else current_price
)

if new_qty < 0 or new_price < 0:

```

```

        raise ValueError(

            "Quantity and price must be non-negative."

        )

    new_status = calculate_status(new_qty)

    cursor.execute("""
        UPDATE hardware
        SET
            quantity = ?,
            unit_price = ?,
            status = ?
        WHERE item_id = ?
    """, (
        new_qty,
        new_price,
        new_status,
        item_id
    ))

    conn.commit()

    logger.info(
        f"Hardware ID {item_id} updated. "
        f"New Qty: {new_qty}, "
        f"New Price: {new_price}"
    )

    return True, "Record updated successfully!"

except ValueError:

```

```

        logger.warning(
            f"Update failed for item ID {item_id}: "
            f"Invalid numeric input."
        )

    return False, (
        "Quantity must be a valid non-negative "
        "integer and price a valid number."
    )

except sqlite3.Error as e:

    logger.error(
        f"Database update error for ID {item_id}: {e}"
    )

    return False, "Database update failed."

finally:

    if conn:
        conn.close()

# =====
# DELETE HARDWARE
# =====

def delete硬件(self, item_id):

    conn = None

```

```
try:

    conn = sqlite3.connect(self.db_name)

    cursor = conn.cursor()

    cursor.execute(
        "SELECT item_name FROM hardware WHERE item_id = ?",
        (item_id,)
    )

    record = cursor.fetchone()

    if not record:

        return False, "Selected record could not be found."

    cursor.execute(
        "DELETE FROM hardware WHERE item_id = ?",
        (item_id,)
    )

    conn.commit()

    logger.info(
        f"Hardware ID {item_id} ({record[0]}) deleted."
    )

    return True, f"{record[0]} was deleted successfully."

except sqlite3.Error as e:

    logger.error(f"Database delete error for ID {item_id}: {e}")
```

```
        return False, "Database delete failed."
```

```
    finally:
```

```
        if conn:
```

```
            conn.close()
```

```
# =====
```

```
# CSV EXPORT
```

```
# =====
```

```
def export_to_csv(
```

```
    self,
```

```
    filename="inventory_report.csv"
```

```
):
```

```
    try:
```

```
        rows = self.fetch_all_hardware()
```

```
        timestamp = datetime.now().strftime(
```

```
            "%Y-%m-%d %H:%M:%S"
```

```
        )
```

```
        total_valuation = 0.0
```

```
        formatted_rows = []
```

```
        for r in rows:
```

```
            (
```

```
        item_id,  
        item_name,  
        category,  
        qty,  
        unit_price,  
        status  
    ) = r
```

```
    item_total = qty * unit_price
```

```
    total_valuation += item_total
```

```
    formatted_rows.append([  
        item_id,  
        item_name,  
        category,  
        qty,  
        f"${unit_price:,.2f}",  
        f"${item_total:,.2f}",  
        status  
    ])
```

```
with open(  
    filename,  
    mode="w",  
    newline="",  
    encoding="utf-8"  
) as file:
```

```
    writer = csv.writer(file)
```

```
writer.writerow([  
    "=====  
    ])
```

```
writer.writerow([  
    "CAMPUS HARDWARE INVENTORY MANAGEMENT REPORT"  
    ])
```

```
writer.writerow([  
    f"Generated On: {timestamp}"  
    ])
```

```
writer.writerow([  
    f"Total Recorded Items: {len(rows)}"  
    ])
```

```
writer.writerow([  
    "=====  
    ])
```

```
writer.writerow([])
```

```
writer.writerow([  
    "Item ID",  
    "Item Name",  
    "Category",  
    "Quantity",  
    "Unit Price",  
    "Total Price",  
    "Status"  
    ])
```

```
writer.writerows(formatted_rows)
```

```
writer.writerow([])
```

```
writer.writerow([  
    "",  
    "",  
    "",  
    "",  
    "GRAND TOTAL ASSET VALUE:",  
    f"${total_valuation:,.2f}",  
    ""  
])
```

```
logger.info(  
    f"CSV Export generated: {filename}. "  
    f"Total Value: ${total_valuation:,.2f}"  
)
```

```
return True, (  
    f"Report successfully exported to "  
    f"{filename}!"  
)
```

```
except Exception as e:
```

```
logger.error(  
    f"CSV Export failed: {e}"  
)
```

```
return False, f"Export failed: {e}"
```

```
# =====
```

```
# 6. LOGIN / REGISTER / RESET WINDOW
```

```
# =====
```

```
class LoginWindow:
```

```
def __init__(self, root, on_success):
```

```
    self.root = root
```

```
    self.on_success = on_success
```

```
    self.controller = InventoryController()
```

```
    self.root.title("Campus Hardware Inventory - Authentication")
```

```
    self.root.geometry("520x640")
```

```
    self.root.resizable(False, False)
```

```
# =====
```

```
# TITLE
```

```
# =====
```

```
tk.Label(
```

```
    root,
```

```
    text="Campus Hardware Inventory",
```

```
    font=("Arial", 18, "bold")
```

```
).pack(pady=(20, 5))
```

```
tk.Label(
```

```
    root,
```

```
        text="User Authentication",  
        font=("Arial", 11)  
    ).pack(pady=(0, 15))
```

```
# =====  
# NOTEBOOK / SEPARATE PANES  
# =====
```

```
self.notebook = ttk.Notebook(root)
```

```
self.login_frame = tk.Frame(self.notebook)  
self.register_frame = tk.Frame(self.notebook)  
self.reset_frame = tk.Frame(self.notebook)
```

```
self.notebook.add(  
    self.login_frame,  
    text=" Login "  
)
```

```
self.notebook.add(  
    self.register_frame,  
    text=" Register "  
)
```

```
self.notebook.add(  
    self.reset_frame,  
    text=" Reset / Unlock Password "  
)
```

```
self.notebook.pack(  
    fill="both",
```

```
        expand=True,  
        padx=25,  
        pady=10  
    )
```

```
self.create_login_pane()  
self.create_register_pane()  
self.create_reset_pane()
```

```
# =====  
# LOGIN PANE  
# =====
```

```
def create_login_pane(self):
```

```
    frame = self.login_frame
```

```
    # Username
```

```
    tk.Label(  
        frame,  
        text="Username:"  
    ).pack(anchor="w", padx=40, pady=(30, 5))
```

```
    self.login_username = tk.Entry(  
        frame,  
        width=38  
    )
```

```
    self.login_username.pack(  
        padx=40,
```

```

        pady=(0, 15)
    )

# Password

tk.Label(
    frame,
    text="Password:"
).pack(anchor="w", padx=40, pady=(0, 5))

self.login_password = tk.Entry(
    frame,
    show="*",
    width=38
)

self.login_password.pack(
    padx=40
)

# =====
# SHOW / HIDE PASSWORD
# =====

self.login_show_password = tk.BooleanVar(
    value=False
)

tk.Checkbutton(
    frame,
    text="Show Password",

```

```
        variable=self.login_show_password,  
        command=self.toggle_login_password  
    ).pack(  
        anchor="w",  
        padx=40,  
        pady=(0, 15)  
    )
```

```
# =====  
# LOGIN BUTTON  
# =====
```

```
tk.Button(  
    frame,  
    text="Login",  
    command=self.handle_login,  
    bg="#4CAF50",  
    fg="white",  
    width=18,  
    height=2  
).pack(pady=10)
```

```
# =====  
# REGISTER PANE  
# =====
```

```
def create_register_pane(self):
```

```
    frame = self.register_frame
```

```
    # Username
```

```
tk.Label(  
    frame,  
    text="Username:"  
)  
.pack(  
    anchor="w",  
    padx=40,  
    pady=(20, 5)  
)
```

```
self.register_username = tk.Entry(  
    frame,  
    width=38  
)
```

```
self.register_username.pack(  
    padx=40,  
    pady=(0, 10)  
)
```

Email

```
tk.Label(  
    frame,  
    text="Email Address:"  
)  
.pack(  
    anchor="w",  
    padx=40,  
    pady=(0, 5)  
)
```

```
self.register_email = tk.Entry(  
    frame,  
    width=38  
)
```

```
self.register_email.pack(  
    padx=40,  
    pady=(0, 3)  
)
```

```
tk.Label(  
    frame,  
    text="Only @gmail.com or @yahoo.com addresses are accepted.",  
    fg="gray",  
    font=("Arial", 8)  
) .pack(  
    anchor="w",  
    padx=40,  
    pady=(0, 10)  
)
```

Role

```
tk.Label(  
    frame,  
    text="Account Role:"  
) .pack(  
    anchor="w",  
    padx=40,  
    pady=(0, 5)  
)
```

```
self.register_role = ttk.Combobox(  
    frame,  
    values=list(VALID_ROLES),  
    state="readonly",  
    width=35  
)
```

```
self.register_role.current(0)
```

```
self.register_role.pack(  
    padx=40,  
    pady=(0, 10)  
)
```

```
# Password
```

```
tk.Label(  
    frame,  
    text="Password:"  
) .pack(  
    anchor="w",  
    padx=40,  
    pady=(0, 5)  
)
```

```
self.register_password = tk.Entry(  
    frame,  
    show="*",  
    width=38  
)
```

```

self.register_password.pack(
    padx=40
)

# =====
# PASSWORD REQUIREMENTS NOTE
# =====

self.register_password_note = tk.Label(
    frame,
    text=PASSWORD_REQUIREMENTS_TEXT,
    fg="red",
    font=("Arial", 9),
    wraplength=360,
    justify="left"
)

self.register_password_note.pack(
    anchor="w",
    padx=40,
    pady=(5, 5)
)

self.register_password.bind(
    "<KeyRelease>",
    self.update_register_password_note
)

# =====
# SHOW / HIDE PASSWORD

```

```
# =====
```

```
self.register_show_password = tk.BooleanVar(  
    value=False  
)
```

```
tk.Checkbutton(  
    frame,  
    text="Show Password",  
    variable=self.register_show_password,  
    command=self.toggle_register_password  
)  
.pack(  
    anchor="w",  
    padx=40,  
    pady=(0, 10)  
)
```

```
# =====
```

```
# REGISTER BUTTON
```

```
# =====
```

```
tk.Button(  
    frame,  
    text="Create Account",  
    command=self.handle_register,  
    bg="#2196F3",  
    fg="white",  
    width=18,  
    height=2  
)  
.pack(pady=10)
```

```

# =====

# RESET / UNLOCK PANE

# =====

def create_reset_pane(self):

    frame = self.reset_frame

    tk.Label(
        frame,
        text="Reset / Unlock your account by submitting a new\n"
            "password. An administrator must approve this\n"
            "request before you can log in again.",
        font=("Arial", 9),
        fg="gray",
        justify="left"
    ).pack(anchor="w", padx=40, pady=(20, 15))

    # Account Username

    tk.Label(
        frame,
        text="Account Username:"
    ).pack(anchor="w", padx=40, pady=(0, 5))

    self.reset_username = tk.Entry(frame, width=38)
    self.reset_username.pack(padx=40, pady=(0, 10))

    # Registered Email

    tk.Label(

```

```

        frame,

        text="Registered Email:"

    ).pack(anchor="w", padx=40, pady=(0, 5))


self.reset_email = tk.Entry(frame, width=38)
self.reset_email.pack(padx=40, pady=(0, 10))


# New Password


tk.Label(

    frame,

    text="Desired New Password:"

).pack(anchor="w", padx=40, pady=(0, 5))


self.reset_new_password = tk.Entry(frame, show="*", width=38)
self.reset_new_password.pack(padx=40)


# =====
# PASSWORD REQUIREMENTS NOTE (RED/GREEN)
# =====


self.reset_password_note = tk.Label(

    frame,

    text=PASSWORD_REQUIREMENTS_TEXT,

    fg="red",

    font=("Arial", 9),

    wraplength=360,

    justify="left"

)


self.reset_password_note.pack(anchor="w", padx=40, pady=(5, 10))

```

```

self.reset_new_password.bind(
    "<KeyRelease>",
    self.update_reset_password_note
)

# Confirm New Password

tk.Label(
    frame,
    text="Confirm New Password:"
).pack(anchor="w", padx=40, pady=(0, 5))

self.reset_confirm_password = tk.Entry(frame, show="*", width=38)
self.reset_confirm_password.pack(padx=40, pady=(0, 10))

# =====
# SHOW / HIDE PASSWORD
# =====

self.reset_show_password = tk.BooleanVar(value=False)

tk.Checkbutton(
    frame,
    text="Show Password",
    variable=self.reset_show_password,
    command=self.toggle_reset_password
).pack(anchor="w", padx=40, pady=(0, 10))

# =====
# SUBMIT BUTTON

```

```
# =====
```

```
tk.Button(  
    frame,  
    text="Submit Reset Request",  
    command=self.handle_reset_submit,  
    bg="#ff9800",  
    fg="white",  
    width=20,  
    height=2  
)
```

```
# =====
```

```
# PASSWORD NOTE - REGISTER
```

```
# =====
```

```
def update_register_password_note(self, event=None):
```

```
    password = self.register_password.get()  
    self._update_password_note(self.register_password_note, password)
```

```
# =====
```

```
# PASSWORD NOTE - RESET
```

```
# =====
```

```
def update_reset_password_note(self, event=None):
```

```
    password = self.reset_new_password.get()  
    self._update_password_note(self.reset_password_note, password)
```

```
@staticmethod
```

```

def _update_password_note(label_widget, password):

    missing = get_password_requirements(password)

    if not password:

        label_widget.config(
            text=PASSWORD_REQUIREMENTS_TEXT,
            fg="red"
        )

    elif missing:

        label_widget.config(
            text="Password is missing:\n• " + "\n• ".join(missing),
            fg="red"
        )

    else:

        label_widget.config(
            text="✓ Password meets all requirements.",
            fg="green"
        )

# =====
# SHOW / HIDE LOGIN PASSWORD
# =====

def toggle_login_password(self):

```

```
if self.login_show_password.get():
```

```
    self.login_password.config(  
        show=""  
    )
```

```
else:
```

```
    self.login_password.config(  
        show="*"  
    )
```

```
# =====  
# SHOW / HIDE REGISTER PASSWORD  
# =====
```

```
def toggle_register_password(self):
```

```
    if self.register_show_password.get():
```

```
        self.register_password.config(  
            show=""  
        )
```

```
    else:
```

```
        self.register_password.config(  
            show="*"  
        )
```

```
# =====
```

```
# SHOW / HIDE RESET PASSWORD
```

```
# =====
```

```
def toggle_reset_password(self):
```

```
    show_char = "" if self.reset_show_password.get() else "*"
```

```
    self.reset_new_password.config(show=show_char)
```

```
    self.reset_confirm_password.config(show=show_char)
```

```
# =====
```

```
# LOGIN
```

```
# =====
```

```
def handle_login(self):
```

```
    user = self.login_username.get().strip()
```

```
    pwd = self.login_password.get()
```

```
    status, msg, role = self.controller.login_user(
```

```
        user,
```

```
        pwd
```

```
    )
```

```
    if status == "success":
```

```
        messagebox.showinfo(
```

```
            "Success",
```

```
            msg
```

```
        )
```

```

self.on_success(user, role)

elif status == "locked":

    # Pressing OK on this prompt then redirects the user
    # straight to the Reset / Unlock Password tab.

    messagebox.showerror(
        "Account Locked",
        msg
    )

    self.reset_username.delete(0, tk.END)
    self.reset_username.insert(0, user)

    self.notebook.select(self.reset_frame)

else:

    messagebox.showerror(
        "Login Failed",
        msg
    )

# =====
# REGISTER
# =====

def handle_register(self):

    user = self.register_username.get().strip()

```

```
email = self.register_email.get().strip()
```

```
pwd = self.register_password.get()
```

```
role = self.register_role.get().strip()
```

```
success, msg = self.controller.register_user(
```

```
    user,
```

```
    email,
```

```
    pwd,
```

```
    role
```

```
)
```

```
if success:
```

```
    messagebox.showinfo(
```

```
        "Registration Successful",
```

```
        msg
```

```
    )
```

```
# Clear registration form
```

```
self.register_username.delete(
```

```
    0,
```

```
    tk.END
```

```
)
```

```
self.register_email.delete(
```

```
    0,
```

```
    tk.END
```

```
)
```

```
self.register_password.delete(
```

```
    0,  
    tk.END  
)
```

```
self.register_role.current(0)
```

```
self.update_register_password_note()
```

```
# Switch back to login tab
```

```
self.notebook.select(  
    self.login_frame  
)
```

```
# Pre-fill username
```

```
self.login_username.delete(  
    0,  
    tk.END  
)
```

```
self.login_username.insert(  
    0,  
    user  
)
```

```
else:
```

```
messagebox.showwarning(  
    "Registration Alert",  
    msg
```

)

```
# =====
```

```
# RESET / UNLOCK SUBMIT
```

```
# =====
```

```
def handle_reset_submit(self):
```

```
    username = self.reset_username.get().strip()
```

```
    email = self.reset_email.get().strip()
```

```
    new_password = self.reset_new_password.get()
```

```
    confirm_password = self.reset_confirm_password.get()
```

```
    success, msg = self.controller.submit_reset_request(
```

```
        username,
```

```
        email,
```

```
        new_password,
```

```
        confirm_password
```

```
    )
```

```
    if success:
```

```
        messagebox.showinfo("Request Submitted", msg)
```

```
        self.reset_username.delete(0, tk.END)
```

```
        self.reset_email.delete(0, tk.END)
```

```
        self.reset_new_password.delete(0, tk.END)
```

```
        self.reset_confirm_password.delete(0, tk.END)
```

```
        self.update_reset_password_note()
```

```
self.notebook.select(self.login_frame)
```

```
else:
```

```
messagebox.showwarning("Reset Request Error", msg)
```

```
# =====
```

```
# 7. INVENTORY WINDOW
```

```
# =====
```

```
class InventoryWindow:
```

```
def __init__(
```

```
    self,
```

```
    root,
```

```
    username,
```

```
    role,
```

```
    on_logout
```

```
):
```

```
    self.root = root
```

```
    self.username = username
```

```
    self.role = role
```

```
    self.on_logout = on_logout
```

```
    self.controller = InventoryController()
```

```
    self.is_admin = (role == ROLE_ADMIN)
```

```
    self.root.title(
```

```
        f"Campus Hardware Inventory - User: {self.username} ({self.role})"
```

```
)
```

```

self.root.geometry(

    "980x780"

)

# =====

# HEADER - ROLE + VALUATION

# =====

header = tk.Frame(self.root, bg="#333")
header.pack(fill="x")

tk.Label(

    header,

    text=f"Logged in as: {self.username} | Role: {self.role}",
    font=("Arial", 11, "bold"),
    bg="#333",
    fg="white",
    pady=8,
    padx=10
).pack(side="left")

self.lbl_valuation = tk.Label(

    header,

    text="Total Asset Value: $0.00",
    font=("Arial", 11, "bold"),
    bg="#333",
    fg="white",
    pady=8,
    padx=10

)

```

```
self.lbl_valuation.pack(side="right")
```

```
# =====
```

```
# MAIN NOTEBOOK
```

```
# =====
```

```
self.notebook = ttk.Notebook(self.root)
```

```
self.notebook.pack(fill="both", expand=True, padx=5, pady=5)
```

```
self.catalog_frame = tk.Frame(self.notebook)
```

```
self.notebook.add(self.catalog_frame, text=" Hardware Catalog ")
```

```
self.build_catalog_tab(self.catalog_frame)
```

```
if self.is_admin:
```

```
    self.approvals_frame = tk.Frame(self.notebook)
```

```
    self.notebook.add(self.approvals_frame, text=" Admin Approvals ")
```

```
    self.build_approvals_tab(self.approvals_frame)
```

```
    self.notebook.bind(
```

```
        "<<NotebookTabChanged>>",
```

```
        self.on_tab_changed
```

```
    )
```

```
else:
```

```
    self.profile_frame = tk.Frame(self.notebook)
```

```
    self.notebook.add(self.profile_frame, text=" My Profile and Security ")
```

```
    self.build_profile_tab(self.profile_frame)
```

```

# =====

# LOGOUT (always visible, outside the notebook)

# =====

frame_bottom = tk.Frame(self.root)
frame_bottom.pack(fill="x", padx=10, pady=(0, 10))

tk.Button(
    frame_bottom,
    text="Logout",
    command=self.logout,
    bg="#dc3545",
    fg="white"
).pack(side="right", padx=5)

self.refresh_data()

# =====

# TAB CHANGE HANDLER (refresh admin approvals)

# =====

def on_tab_changed(self, event=None):

    if self.is_admin and self.notebook.select() == str(self.approvals_frame):
        self.refresh_approvals()

# =====

# BUILD: HARDWARE CATALOG TAB

# =====

```

```

def build_catalog_tab(self, parent):

    # -----
    # ADD EQUIPMENT FORM (ADMIN ONLY)
    # -----

    if self.is_admin:

        frame_form = tk.LabelFrame(
            parent,
            text="Add New Equipment",
            padx=10,
            pady=10
        )

        frame_form.pack(fill="x", padx=10, pady=5)

        inner_form = tk.Frame(frame_form)
        inner_form.pack(side="left", padx=5)

        tk.Label(inner_form, text="Name:").grid(
            row=0, column=0, padx=5, pady=5, sticky="e"
        )

        self.ent_name = tk.Entry(inner_form, width=20)
        self.ent_name.grid(row=0, column=1, padx=5, pady=5)

        tk.Label(inner_form, text="Category:").grid(
            row=0, column=2, padx=5, pady=5, sticky="e"
        )

```

```
self.ent_cat = tk.Entry(inner_form, width=20)
self.ent_cat.grid(row=0, column=3, padx=5, pady=5)
```

```
tk.Label(inner_form, text="Quantity:").grid(
    row=1, column=0, padx=5, pady=5, sticky="e"
)
```

```
self.ent_qty = tk.Entry(inner_form, width=20)
self.ent_qty.grid(row=1, column=1, padx=5, pady=5)
```

```
tk.Label(inner_form, text="Unit Price ($)").grid(
    row=1, column=2, padx=5, pady=5, sticky="e"
)
```

```
self.ent_price = tk.Entry(inner_form, width=20)
self.ent_price.grid(row=1, column=3, padx=5, pady=5)
```

```
tk.Button(
    inner_form,
    text="Save Item",
    command=self.add_item,
    bg="#4CAF50",
    fg="white",
    padx=10
).grid(row=0, column=4, rowspan=2, padx=15, pady=5, sticky="nsew")
```

```
frame_legend = tk.LabelFrame(
    frame_form,
    text="Status Legend",
    padx=10,
    pady=5
```

)

```
frame_legend.pack(side="right", padx=15, pady=5)
```

```
tk.Label(  
    frame_legend, text="In Stock (> 5)",  
    bg="#d4edda", fg="#155724", font=("Arial", 8, "bold"),  
    relief="solid", bd=1, anchor="center"  
) .grid(row=0, column=0, padx=5, pady=2, sticky="ew")
```

```
tk.Label(  
    frame_legend, text="Low Stock (1-5)",  
    bg="#fff3cd", fg="#856404", font=("Arial", 8, "bold"),  
    relief="solid", bd=1, anchor="center"  
) .grid(row=1, column=0, padx=5, pady=2, sticky="ew")
```

```
tk.Label(  
    frame_legend, text="Out of Stock (0)",  
    bg="#f8d7da", fg="#721c24", font=("Arial", 8, "bold"),  
    relief="solid", bd=1, anchor="center"  
) .grid(row=2, column=0, padx=5, pady=2, sticky="ew")
```

```
# -----  
# FILTER / SEARCH BAR (ALL ROLES)  
# -----
```

```
frame_filter = tk.LabelFrame(  
    parent,  
    text="Filter & Search",  
    padx=10,  
    pady=10
```

```
)
```

```
frame_filter.pack(fill="x", padx=10, pady=5)
```

```
tk.Label(frame_filter, text="Category:").pack(side="left", padx=(5, 5))
```

```
self.category_filter = ttk.Combobox(
```

```
    frame_filter,
```

```
    state="readonly",
```

```
    width=22
```

```
)
```

```
self.category_filter.pack(side="left", padx=(0, 20))
```

```
self.category_filter.bind("<<ComboboxSelected>>", lambda e: self.refresh_data())
```

```
tk.Label(frame_filter, text="Search Name:").pack(side="left", padx=(5, 5))
```

```
self.search_entry = tk.Entry(frame_filter, width=30)
```

```
self.search_entry.pack(side="left", padx=(0, 5))
```

```
self.search_entry.bind("<KeyRelease>", lambda e: self.refresh_data())
```

```
tk.Button(
```

```
    frame_filter,
```

```
    text="Clear Filters",
```

```
    command=self.clear_filters
```

```
).pack(side="left", padx=10)
```

```
# -----
```

```
# DATA TABLE
```

```
# -----
```

```

frame_table = tk.Frame(parent)

frame_table.pack(fill="both", expand=True, padx=10, pady=5)


scroll_y = tk.Scrollbar(frame_table, orient=tk.VERTICAL)


self.tree = ttk.Treeview(
    frame_table,
    columns=("ID", "Name", "Category", "Qty", "Price", "Status"),
    show="headings",
    yscrollcommand=scroll_y.set
)

scroll_y.config(command=self.tree.yview)
scroll_y.pack(side=tk.RIGHT, fill=tk.Y)


for col in self.tree["columns"]:
    self.tree.heading(col, text=col)
    self.tree.column(col, width=120, anchor="center")


self.tree.tag_configure("In Stock", background="#d4edda")
self.tree.tag_configure("Low Stock", background="#fff3cd")
self.tree.tag_configure("Out of Stock", background="#f8d7da")


self.tree.pack(fill="both", expand=True)


# -----
# UPDATE / DELETE FRAME (ADMIN ONLY)
# -----


if self.is_admin:

```

```

frame_update = tk.LabelFrame(
    parent,
    text="Update / Delete Selected Record",
    padx=10,
    pady=5
)

frame_update.pack(fill="x", padx=10, pady=5)

inner_update = tk.Frame(frame_update)
inner_update.pack()

tk.Label(inner_update, text="New Quantity:").grid(
    row=0, column=0, padx=5, pady=5, sticky="e"
)

self.ent_up_qty = tk.Entry(inner_update, width=18)
self.ent_up_qty.grid(row=0, column=1, padx=5, pady=5)

tk.Label(inner_update, text="New Unit Price ($)").grid(
    row=0, column=2, padx=5, pady=5, sticky="e"
)

self.ent_up_price = tk.Entry(inner_update, width=18)
self.ent_up_price.grid(row=0, column=3, padx=5, pady=5)

tk.Button(
    inner_update,
    text="Update Record",
    command=self.update_item,
    bg="#2196F3",

```

```
        fg="white",
        padx=10
    ).grid(row=0, column=4, padx=15, pady=5, sticky="ew")
```

```
tk.Button(
    inner_update,
    text="Delete Record",
    command=self.delete_item,
    bg="#dc3545",
    fg="white",
    padx=10
).grid(row=0, column=5, padx=5, pady=5, sticky="ew")
```

```
# -----
# EXPORT (ALL ROLES)
# -----
```

```
frame_controls = tk.Frame(parent)
frame_controls.pack(fill="x", padx=10, pady=10)
```

```
tk.Button(
    frame_controls,
    text="Export Inventory to CSV",
    command=self.export_csv,
    bg="#17a2b8",
    fg="white"
).pack(side="left", padx=5)
```

```
# =====
# BUILD: MY PROFILE AND SECURITY TAB (USER)
# =====
```

```
def build_profile_tab(self, parent):
```

```
    profile_row = self.controller.get_user_profile(self.username)
```

```
    acc_username, acc_email, acc_role = profile_row if profile_row else (self.username, "", self.role)
```

```
    frame_overview = tk.LabelFrame(
```

```
        parent,
```

```
        text="Account Overview",
```

```
        padx=15,
```

```
        pady=15
```

```
    )
```

```
    frame_overview.pack(fill="x", padx=20, pady=20)
```

```
    tk.Label(frame_overview, text=f"Username: {acc_username}", font=("Arial", 10)).grid(
```

```
        row=0, column=0, sticky="w", pady=3
```

```
    )
```

```
    tk.Label(frame_overview, text=f"Registered Email: {acc_email}", font=("Arial", 10)).grid(
```

```
        row=1, column=0, sticky="w", pady=3
```

```
    )
```

```
    tk.Label(frame_overview, text=f"Account Role: {acc_role}", font=("Arial", 10)).grid(
```

```
        row=2, column=0, sticky="w", pady=3
```

```
    )
```

```
    frame_security = tk.LabelFrame(
```

```
        parent,
```

```
        text="Update Password Directly",
```

```
    padx=15,  
    pady=15  
)
```

```
frame_security.pack(fill="x", padx=20, pady=(0, 20))
```

```
tk.Label(frame_security, text="New Password:").grid(  
    row=0, column=0, sticky="e", padx=5, pady=5  
)
```

```
self.profile_new_password = tk.Entry(frame_security, show="*", width=30)  
self.profile_new_password.grid(row=0, column=1, padx=5, pady=5)
```

```
tk.Label(frame_security, text="Confirm Password:").grid(  
    row=1, column=0, sticky="e", padx=5, pady=5  
)
```

```
self.profile_confirm_password = tk.Entry(frame_security, show="*", width=30)  
self.profile_confirm_password.grid(row=1, column=1, padx=5, pady=5)
```

```
self.profile_password_note = tk.Label(  
    frame_security,  
    text=PASSWORD_REQUIREMENTS_TEXT,  
    fg="red",  
    font=("Arial", 9),  
    wraplength=360,  
    justify="left"  
)
```

```
self.profile_password_note.grid(row=2, column=0, colspan=2, sticky="w", padx=5, pady=(5,  
5))
```

```
self.profile_new_password.bind(  
    "<KeyRelease>",  
    self.update_profile_password_note  
)
```

```
self.profile_show_password = tk.BooleanVar(value=False)
```

```
tk.Checkbutton(  
    frame_security,  
    text="Show Password",  
    variable=self.profile_show_password,  
    command=self.toggle_profile_password  
)grid(row=3, column=0, columnspan=2, sticky="w", padx=5, pady=(0, 10))
```

```
tk.Button(  
    frame_security,  
    text="Update Password",  
    command=self.handle_profile_password_update,  
    bg="#2196F3",  
    fg="white",  
    padx=10  
)grid(row=4, column=0, columnspan=2, pady=5)
```

```
def update_profile_password_note(self, event=None):
```

```
    password = self.profile_new_password.get()  
    LoginWindow._update_password_note(self.profile_password_note, password)
```

```
def toggle_profile_password(self):
```

```

show_char = "" if self.profile_show_password.get() else "*"

self.profile_new_password.config(show=show_char)
self.profile_confirm_password.config(show=show_char)

def handle_profile_password_update(self):

    new_password = self.profile_new_password.get()
    confirm_password = self.profile_confirm_password.get()

    success, msg = self.controller.update_password_direct(
        self.username,
        new_password,
        confirm_password
    )

    if success:

        messagebox.showinfo("Success", msg)

        self.profile_new_password.delete(0, tk.END)
        self.profile_confirm_password.delete(0, tk.END)
        self.update_profile_password_note()

    else:

        messagebox.showerror("Password Update Failed", msg)

# =====
# BUILD: ADMIN APPROVALS TAB
# =====

```

```

def build_approvals_tab(self, parent):

    tk.Label(
        parent,
        text="Pending Password Reset / Unlock Requests",
        font=("Arial", 12, "bold")
    ).pack(anchor="w", padx=15, pady=(15, 10))

    frame_table = tk.Frame(parent)
    frame_table.pack(fill="both", expand=True, padx=15, pady=5)

    scroll_y = tk.Scrollbar(frame_table, orient=tk.VERTICAL)

    self.approvals_tree = ttk.Treeview(
        frame_table,
        columns=("Request ID", "Username", "Email", "Timestamp"),
        show="headings",
        yscrollcommand=scroll_y.set
    )

    scroll_y.config(command=self.approvals_tree.yview)
    scroll_y.pack(side=tk.RIGHT, fill=tk.Y)

    for col in self.approvals_tree["columns"]:
        self.approvals_tree.heading(col, text=col)
        self.approvals_tree.column(col, width=150, anchor="center")

    self.approvals_tree.pack(fill="both", expand=True)

    frame_buttons = tk.Frame(parent)

```

```
frame_buttons.pack(fill="x", padx=15, pady=15)
```

```
tk.Button(  
    frame_buttons,  
    text="Approve Reset",  
    command=self.handle_approve_reset,  
    bg="#28a745",  
    fg="white",  
    width=18,  
    height=2  
) .pack(side="left", padx=10)
```

```
tk.Button(  
    frame_buttons,  
    text="Reject Reset",  
    command=self.handle_reject_reset,  
    bg="#dc3545",  
    fg="white",  
    width=18,  
    height=2  
) .pack(side="left", padx=10)
```

```
self.refresh_approvals()
```

```
def refresh_approvals(self):
```

```
    for row in self.approvals_tree.get_children():  
        self.approvals_tree.delete(row)
```

```
    for r in self.controller.get_pending_reset_requests():  
        self.approvals_tree.insert("", tk.END, values=r)
```

```
def handle_approve_reset(self):
```

```
    selected = self.approvals_tree.selection()
```

```
    if not selected:
```

```
        messagebox.showwarning("Selection Error", "Please select a request to approve.")
```

```
        return
```

```
    request_id = self.approvals_tree.item(selected, "values")[0]
```

```
    success, msg = self.controller.approve_reset_request(request_id)
```

```
    if success:
```

```
        messagebox.showinfo("Request Approved", msg)
```

```
    else:
```

```
        messagebox.showerror("Error", msg)
```

```
    self.refresh_approvals()
```

```
def handle_reject_reset(self):
```

```
    selected = self.approvals_tree.selection()
```

```
    if not selected:
```

```
        messagebox.showwarning("Selection Error", "Please select a request to reject.")
```

```
        return
```

```
    request_id = self.approvals_tree.item(selected, "values")[0]
```

```
    success, msg = self.controller.reject_reset_request(request_id)
```

```

if success:

    messagebox.showinfo("Request Rejected", msg)
else:

    messagebox.showerror("Error", msg)


self.refresh_approvals()


# =====
# CATEGORY FILTER HELPERS
# =====


def refresh_categories(self):

    categories = ["All Categories"] + self.controller.get_distinct_categories()

    current = self.category_filter.get()

    self.category_filter["values"] = categories

    if current in categories:

        self.category_filter.set(current)
    else:

        self.category_filter.set("All Categories")


def clear_filters(self):

    self.search_entry.delete(0, tk.END)

    self.category_filter.set("All Categories")

    self.refresh_data()

```

```

# =====

# REFRESH TABLE

# =====

def refresh_data(self):

    self.refresh_categories()

    for row in self.tree.get_children():
        self.tree.delete(row)

    category = self.category_filter.get() or "All Categories"
    search_text = self.search_entry.get() if hasattr(self, "search_entry") else ""

    rows = self.controller.fetch_filtered_hardware(category, search_text)

    for r in rows:

        status = r[5]

        self.tree.insert(
            "",
            tk.END,
            values=r,
            tags=(status,)
        )

    total_val = self.controller.get_total_asset_value()

    self.lbl_valuation.config(
        text=f"Total Asset Value: ${total_val:,.2f}"
    )

```

)

```
# =====
```

```
# ADD ITEM
```

```
# =====
```

```
def add_item(self):
```

```
    name = self.ent_name.get().strip()
```

```
    cat = self.ent_cat.get().strip()
```

```
    qty = self.ent_qty.get().strip()
```

```
    price = self.ent_price.get().strip()
```

```
    success, msg = self.controller.add_hardware(name, cat, qty, price)
```

```
    if success:
```

```
        messagebox.showinfo("Success", msg)
```

```
        self.ent_name.delete(0, tk.END)
```

```
        self.ent_cat.delete(0, tk.END)
```

```
        self.ent_qty.delete(0, tk.END)
```

```
        self.ent_price.delete(0, tk.END)
```

```
        self.refresh_data()
```

```
    else:
```

```
        messagebox.showwarning("Duplicate or Invalid Entry", msg)
```

```
# =====
```

```
# UPDATE ITEM
```

```
# =====
```

```
def update_item(self):
```

```
    selected_item = self.tree.selection()
```

```
    if not selected_item:
```

```
        messagebox.showwarning(
```

```
            "Selection Error",
```

```
            "Please select a row from the table to update!"
```

```
        )
```

```
        return
```

```
    item_values = self.tree.item(selected_item, "values")
```

```
    item_id = item_values[0]
```

```
    qty_str = self.ent_up_qty.get().strip()
```

```
    price_str = self.ent_up_price.get().strip()
```

```
    success, msg = self.controller.update_hardware(item_id, qty_str, price_str)
```

```
    if success:
```

```
        messagebox.showinfo("Success", msg)
```

```
        self.ent_up_qty.delete(0, tk.END)
```

```
        self.ent_up_price.delete(0, tk.END)
```

```
        self.refresh_data()
```

else:

 messagebox.showerror("Error", msg)

=====

DELETE ITEM

=====

def delete_item(self):

 selected_item = self.tree.selection()

 if not selected_item:

 messagebox.showwarning(

 "Selection Error",

 "Please select a row from the table to delete!"

)

 return

 item_values = self.tree.item(selected_item, "values")

 item_id = item_values[0]

 item_name = item_values[1]

 confirmed = messagebox.askyesno(

 "Confirm Delete",

 f"Are you sure you want to delete '{item_name}'? This cannot be undone."

)

 if not confirmed:

 return

```
success, msg = self.controller.delete硬件(item_id)
```

```
if success:
```

```
    messagebox.showinfo("Deleted", msg)
```

```
else:
```

```
    messagebox.showerror("Error", msg)
```

```
self.refresh_data()
```

```
# =====
```

```
# EXPORT CSV
```

```
# =====
```

```
def export_csv(self):
```

```
    success, msg = self.controller.export_to_csv()
```

```
if success:
```

```
    messagebox.showinfo("Export Successful", msg)
```

```
else:
```

```
    messagebox.showerror("Export Failed", msg)
```

```
# =====
```

```
# LOGOUT
```

```
# =====
```

```
def logout(self):
```

```
    logger.info(f'User '{self.username}' logged out.")
```

```
self.on_logout()
```

```
# =====
```

```
# 8. APP LAUNCHER / WINDOW MANAGER
```

```
# =====
```

```
def launch_main_app(username, role):
```

```
    for widget in root.winfo_children():
```

```
        widget.destroy()
```

```
    InventoryWindow(
```

```
        root,
```

```
        username,
```

```
        role,
```

```
        on_logout=launch_login
```

```
    )
```

```
def launch_login():
```

```
    for widget in root.winfo_children():
```

```
        widget.destroy()
```

```
    LoginWindow(
```

```
        root,
```

```
        on_success=launch_main_app
```

```
    )
```

```
# =====
```

9. PROGRAM ENTRY POINT

=====

if __name__ == "__main__":

init_db()

root = tk.Tk()

launch_login()

root.mainloop()